

Título del Proyecto

Respaldar.gio

Autor:

Oscar Giovanni Fuentes Campos

Versiones terminadas:

0.0, 1.0, 2.0,

Correo:

giovannifuentesbdg@gmail.com

Índice

Índice.....	1
Introducción	3
Planteamiento del problema	3
Objetivo general	3
Objetivos específicos	3
Alcance	4
Metodología.....	4
Metodología de desarrollo Ágil Scrum.....	4
Roles en Scrum adaptados a un solo desarrollador	5
Scrum Master	5
Development Team (Equipo de desarrollo).....	5
Product Owner.....	5
Análisis y levantamiento de requisitos	5
El levantamiento de requisitos se realizó tomando en cuenta:	6
Requisitos funcionales:	6
Requisitos no funcionales:	6
Historias de usuario	7
Base Original.....	7
Adicionado después	7
Product Backlog	7
Sprint.....	7
Diseño de interfaces	8
Diseño de la arquitectura	9
PowerShell.....	9
Batch.....	10
Bash (Bourne Again Shell)	11
Python.....	11
Visual Studio Code.....	11
Resultados.....	12
1. Integración inicial	12
2. Requerimientos del sistema.....	12
3. Funcionalidades adicionadas posteriormente	12
Seguridad.....	13
Usabilidad	13

Disponibilidad.....	13
Creación del proyecto en Visual Studio Code.....	13
Sprint 1 – Planificación e Investigación	14
Sprint 2 – Diseño de la Aplicación	14
Sprint 3 – Desarrollo e Implementación.....	15
Sprint 4 – Pruebas y Optimización	15
Conclusiones y Recomendaciones	15
Conclusiones.....	15
Recomendaciones.....	16
Problemas Dentro de la Programación	16
Modificaciones	17
Resultados.....	18
Recomendaciones Externas.....	19
Referencias Bibliográficas	21

Introducción

Debemos crear un sistema que nos ayude a vaciar una PC, tomando en cuenta las carpetas donde el usuario normalmente guarda sus documentos.

Esta necesidad surge porque, cuando un técnico o ingeniero desea formatear un equipo, habitualmente debe revisar carpeta por carpeta, lo cual genera retrasos al intentar rescatar la información del cliente o usuario.

Ya sea que la PC sea muy lenta o presente otros problemas, buscamos la forma más eficiente de extraer la información. Por lo tanto, el sistema tendrá precisamente ese objetivo.

Planteamiento del problema

De forma personal, como estudiante de Sistemas Computacionales, me he enfrentado a un problema constante: la memoria de mi equipo principal (una laptop) se satura con rapidez, por lo que al menos cada cuatrimestre debo formatearlo. Con tantas instalaciones y aplicaciones, el sistema comienza a alentarse, y resulta indispensable rescatar los datos antes de proceder.

Además, en ocasiones surge la necesidad de formatear la PC de algún familiar o amigo, quienes requieren que sus datos permanezcan intactos. Sin embargo, la mayoría nunca ha dado mantenimiento a sus equipos, lo que provoca que se vuelvan lentos y, en consecuencia, sea difícil recuperar la información de manera rápida y eficiente.

Objetivo general

Desarrollar un sistema ligero que permita respaldar todos los documentos de una PC, utilizando la menor cantidad de recursos posible.

Objetivos específicos

1. Identificar las carpetas más utilizadas por los usuarios para almacenar documentos y archivos importantes.
2. Diseñar un sistema que localice y organice automáticamente la información a respaldar.
3. Implementar un mecanismo de copia que consuma la menor cantidad posible de recursos del sistema.

4. Garantizar que el respaldo se realice de manera rápida y segura, evitando la pérdida de información.
5. Probar y evaluar el sistema en distintos equipos para verificar su eficiencia y confiabilidad.

Alcance

El sistema estará enfocado en respaldar de manera automática los documentos almacenados en una PC, priorizando las carpetas donde comúnmente los usuarios guardan su información. Se busca que el software sea ligero, fácil de utilizar y que requiera la menor cantidad de recursos posible, de modo que pueda ejecutarse incluso en equipos con bajo rendimiento.

El proyecto abarcará el análisis, diseño, desarrollo y pruebas del sistema en diferentes computadoras, con el fin de asegurar su correcto funcionamiento en diversos entornos. Sin embargo, no contempla la recuperación de archivos dañados, eliminados permanentemente ni la implementación de sistemas avanzados de almacenamiento en la nube.

Los respaldos se realizarán directamente en una memoria USB o en un disco duro externo, donde se almacenará la información extraída del sistema.

Con este proyecto se busca hacer más eficiente el proceso de respaldo de archivos, ya que en muchas ocasiones es necesario respaldar equipos saturados o desordenados, donde incluso se desconoce qué información se encuentra almacenada. Además, el sistema permitirá organizar los archivos respaldados por tipo de extensión, lo cual facilitará la identificación y clasificación de la información disponible.

Metodología

En este apartado se describe el enfoque que se ha decidido tomar a lo largo del desarrollo de la aplicación. La metodología seleccionada permitirá llevar un control organizado de cada etapa del proyecto, asegurando que se cumplan los objetivos planteados y que el sistema funcione de acuerdo con los requerimientos establecidos.

Metodología de desarrollo Ágil Scrum

Scrum es un marco de trabajo para el desarrollo y mantenimiento de software que emplea técnicas y procesos basados en la organización de equipos y roles específicos. Su implementación se sustenta en tres pilares fundamentales: **transparencia, inspección y adaptación.**

La metodología Scrum es ágil y flexible, lo que permite gestionar el desarrollo de software de manera eficiente, con el principal objetivo de **maximizar el retorno de la inversión realizada**. Además, se adapta especialmente a entornos con pocos recursos, lo que facilita su aplicación incluso cuando el proyecto es desarrollado por una sola persona.

Roles en Scrum adaptados a un solo desarrollador

En este proyecto individual, los tres roles de Scrum se unifican en una sola persona. La diferencia es que yo asumiré la responsabilidad de planear, organizar y desarrollar todas las etapas del sistema.

Scrum Master

Es el responsable de asegurar que las prácticas de Scrum se apliquen de forma correcta. Define qué se va a desarrollar y cuáles son las prioridades del proyecto. También funge como representante del proyecto, facilitando, guiando, eliminando obstáculos y asegurando que las prácticas de Scrum se mantengan durante el proceso

Development Team (Equipo de desarrollo)

Son quienes programan, diseñan, prueban y entregan el producto. Deben estar estructurados y tener la capacidad de gestionar el trabajo de manera autónoma, comprendiendo claramente los requerimientos establecidos. En este caso, yo desempeño ese rol, desarrollando y validando cada funcionalidad.

Product Owner

Es el dueño del producto, responsable de decidir qué debe hacer el programa. Tiene como objetivo maximizar el valor del trabajo realizado y es quien tiene la última palabra en la gestión de prioridades y funcionalidades a implementar.

Persona/Entidad	Rol
Oscar Giovanni Fuentes Campos	Product Owner
Oscar Giovanni Fuentes Campos	Scrum Master
Oscar Giovanni Fuentes Campos	Devolpment Team

Análisis y levantamiento de requisitos

Para lograr un buen funcionamiento del sistema, se realizó un análisis con el fin de identificar los requerimientos necesarios para generar esta aplicación. Este proceso incluyó la revisión de problemas comunes al intentar recuperar datos de un usuario, considerando que no todas las personas cuentan con computadoras de altos recursos, o que sus equipos son antiguos y solo disponen de los recursos mínimos para un funcionamiento normal.

El levantamiento de requisitos se realizó tomando en cuenta:

- La necesidad de respaldar de forma rápida y automática los documentos más importantes del usuario.
- La importancia de que el sistema sea ligero y consuma pocos recursos, para poder ejecutarse en equipos con bajo rendimiento.
- La facilidad de uso, de modo que cualquier persona pueda realizar el respaldo sin conocimientos técnicos avanzados.
- La capacidad de almacenar los respaldos directamente en una memoria USB o en un disco duro externo.
- La posibilidad de organizar los archivos respaldados por tipo de extensión, facilitando su identificación posterior.

A partir de este análisis se definieron los **requisitos funcionales y no funcionales**, que se detallan a continuación:

Requisitos funcionales:

1. El sistema debe identificar automáticamente las carpetas comunes donde los usuarios guardan documentos.
2. El sistema debe respaldar los archivos seleccionados en un dispositivo externo (USB o disco duro).
3. El sistema debe permitir organizar los archivos respaldados por tipo de extensión.
4. El sistema debe mostrar al usuario el progreso y el resultado del respaldo.

Requisitos no funcionales:

1. El sistema debe ser ligero y consumir una cantidad mínima de recursos.
2. El sistema debe funcionar en equipos con especificaciones bajas.
3. El sistema debe contar con una interfaz sencilla y fácil de usar.
4. El tiempo de respaldo debe ser eficiente en comparación con una copia manual.

Historias de usuario

Las historias de usuario son pequeñas descripciones de los requerimientos del cliente. En este caso, representan lo que esperamos que haga el programa, marcando las bases de su funcionamiento.

Base Original

- Ser ligero.
- Multiplataforma.
- Recuperar archivos del usuario.
- Recuperar archivos por extensión.
- Recuperar archivos de XAMPP.
- Guardar los respaldos en otra memoria.
- Hacer respaldos de otras memorias.
- Eliminar respaldos anteriores.

Adicionado después

- Comprimir las carpetas en formato .zip
- Modo “disco externo recuperado”: al montar o conectar un disco externo (USB/HDD retirado de otra PC), el sistema buscará archivos no pertenecientes al sistema operativo.
- Uso de la red interna para extraer archivos de otra PC, con un modo servidor y un modo usuario.

Product Backlog

La primera parte del flujo de trabajo en Scrum es el **Product Backlog**, una lista ordenada que contiene los requisitos del sistema. Debe ser breve, claro y priorizado según la importancia de cada función.

El **Product Owner** es responsable de mantener actualizado el backlog, así como de ordenar y modificar los elementos ante cualquier cambio en los requerimientos o necesidades del proyecto.

Sprint

Los **Sprint** son un conjunto de requisitos o tareas que deben cumplirse en un tiempo determinado. En cada Sprint se seleccionan características específicas del

producto, levantando los requisitos necesarios y desarrollando una parte del proyecto.

En este caso, se trabajará con **4 Sprint** para completar el sistema.

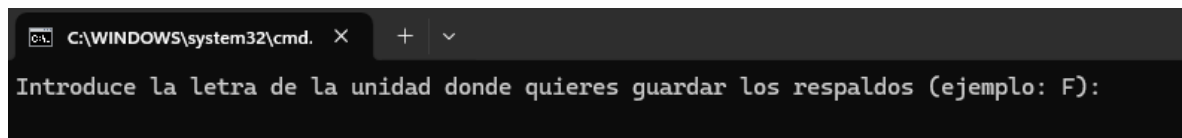
Los Sprint forman parte de la metodología **Scrum**, lo que permite tener un mayor control sobre cada entregable y sobre el progreso del desarrollo. Una de sus ventajas es que algunos Sprint pueden **empalmarse o ejecutarse en paralelo**, lo que ayuda a que el proyecto avance con mayor fluidez y rapidez.

Diseño de interfaces

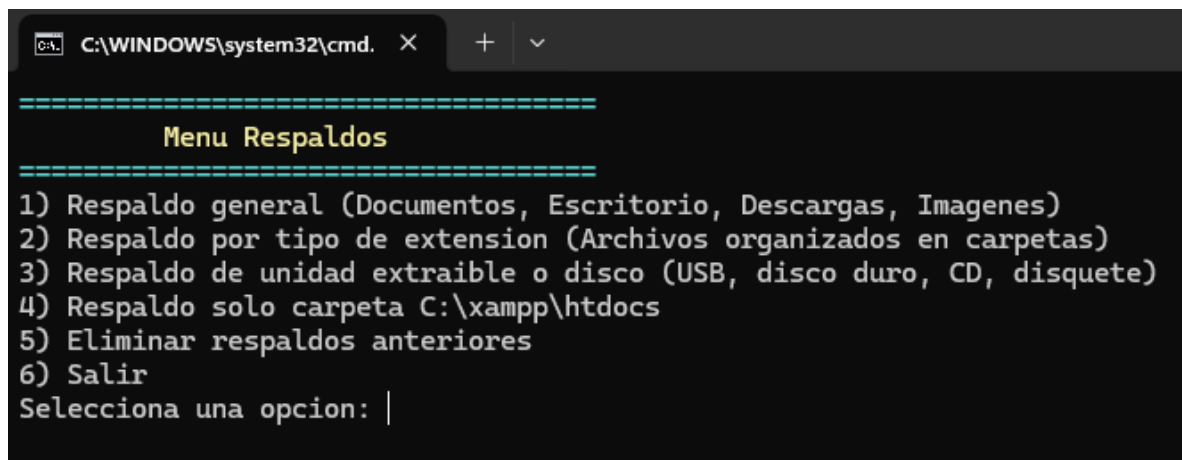
Las interfaces del sistema son los entornos que permiten al usuario interactuar y controlar la aplicación de la manera más efectiva posible. Se busca que la interacción sea **eficiente, clara y sencilla**, facilitando la navegación dentro del programa.

Una de las ventajas de haber programado en **PowerShell y Bash** es que la interfaz resultante es muy simple, lo que permite una ejecución rápida y ligera. El principal reto al trabajar en **Python** consiste en mantener la interfaz igual de **ligera y eficiente**, ya que, si se sobrecarga con demasiados elementos gráficos, podría afectar el rendimiento. La idea principal es que el sistema se mantenga **fácil de usar y con bajo consumo de recursos**.

Prototipo de interfaces



```
C:\WINDOWS\system32\cmd. X + v
Introduce la letra de la unidad donde quieres guardar los respaldos (ejemplo: F):
```



```
C:\WINDOWS\system32\cmd. X + v

=====
Menu Respaldos
=====
1) Respaldo general (Documentos, Escritorio, Descargas, Imagenes)
2) Respaldo por tipo de extension (Archivos organizados en carpetas)
3) Respaldo de unidad extraible o disco (USB, disco duro, CD, disquete)
4) Respaldo solo carpeta C:\xampp\htdocs
5) Eliminar respaldos anteriores
6) Salir
Selecciona una opcion: |
```

Diseño de la arquitectura

Para la arquitectura del sistema se utilizó un **boceto básico**, dado que se trata de un programa sencillo. El flujo principal se compone de los siguientes elementos:

1. **Usuario** → Interactúa con el programa para iniciar el proceso de respaldo.
2. **Disco duro o memoria externa** → Representa el destino donde se guardará la información extraída del sistema.
3. **Programa** → Se encarga de identificar, recopilar y organizar los archivos que deben ser respaldados.
4. **Datos respaldados** → Resultado final, que corresponde a los archivos almacenados de manera segura en el dispositivo externo.

Este modelo simple permite visualizar de forma clara el proceso de interacción y flujo de datos dentro del sistema.



PowerShell

PowerShell es una solución de automatización de tareas multiplataforma que combina un Shell de línea de comandos, un lenguaje de scripting y un marco de administración de configuración.

Se eligió este lenguaje de programación porque es uno de los que se ejecuta más rápido de manera interna en Windows y puede usarse también como multiplataforma. Sin embargo, uno de los retos que encontramos fue la diferencia

estructural entre Linux y Windows, lo que dificultó adaptarlo de una forma más segura y correcta.

Por esta razón, la primera versión del sistema se desarrolló específicamente para Windows, con el objetivo de comprender su funcionamiento y comenzar a usarlo de manera eficiente. Además, se eligió Windows porque es el sistema operativo más utilizado en hogares y probablemente sea el único requerido por la mayoría de los usuarios.



Batch

Los archivos Batch son aplicaciones para Windows que pueden ser creadas por los propios usuarios para realizar una gran variedad de tareas.

Son simples archivos de texto con la extensión .CMD o .BAT, que al ejecutarse (haciendo doble clic sobre ellos) ejecutan sus instrucciones en la Consola de CMD o Símbolo del sistema.

En este proyecto se tuvo que utilizar un archivo .bat porque PowerShell tiene niveles de seguridad que bloqueaban nuestro programa, detectándolo como un acceso no autorizado. Con el archivo .bat, se logró otorgar los permisos necesarios para ejecutar el script sin restricciones.

Se intentó reescribir el script de PowerShell para evitar este bloqueo, pero cada intento produjo el mismo error. Tras investigar, se determinó que el uso del archivo .bat era la opción más factible y segura para ejecutar esta primera versión del sistema.



Bash (Bourne Again Shell)

Bash es uno de los intérpretes de comandos más utilizados en entornos Unix y Linux. Se ha convertido en una herramienta fundamental para administradores de sistemas, desarrolladores y usuarios avanzados que desean interactuar con sus sistemas operativos de manera eficiente.

Se utilizó Bash para desarrollar la segunda versión del sistema, que busca ser más multiplataforma, permitiendo su ejecución en Linux y macOS. Inicialmente se implementó como prototipo para comprender cómo funcionan las carpetas y la estructura de archivos en Linux. Además, se eligió este lenguaje debido a su rapidez y facilidad para ejecutar scripts en estos sistemas operativos.



Python

Python es un lenguaje de programación de propósito general, de alto nivel, interpretado y de código abierto, reconocido por su sintaxis limpia y legible, similar al lenguaje natural, lo que lo hace ideal para principiantes. Es muy versátil y se utiliza en una amplia gama de aplicaciones, como el desarrollo web (back-end), la ciencia de datos, el aprendizaje automático, la automatización de tareas (scripting), entre otras.

Se utilizó Python por su simplicidad y facilidad de adaptación, lo que permitió desarrollar nuestro programa de manera multiplataforma. Además, Python cuenta con múltiples extensiones y bibliotecas que facilitan la implementación de funcionalidades adicionales y simplifican el trabajo de desarrollo.



Visual Studio Code

Visual Studio es un editor de código fuente que ofrece múltiples funcionalidades al momento de trabajar, incluyendo la compatibilidad con diferentes lenguajes de programación. Es multiplataforma y cuenta con plugins que permiten adaptar actualizaciones y mejorar la productividad durante el desarrollo.

Se decidió utilizar Visual Studio principalmente por sus múltiples extensiones, que facilitan la programación y la integración de herramientas adicionales. Además, permite abrir espacios de trabajo organizados, lo que hace que la programación sea más sencilla y eficiente.



Resultados

A lo largo del desarrollo del programa se implementaron distintas funcionalidades organizadas en **4 Sprint**, con el objetivo de cumplir los requerimientos iniciales y adicionar mejoras durante el proceso.

1. Integración inicial

En el primer Sprint se elaboró un listado de los aspectos que servirían como base del sistema:

- Ser ligero.
- Ser multiplataforma.
- Recuperar archivos del usuario.

2. Requerimientos del sistema

En los siguientes Sprint se trabajó en que el sistema pudiera ejecutarse incluso en computadoras con **mínimos recursos**, incorporando las siguientes funciones:

- Recuperar archivos por extensión.
- Recuperar archivos de XAMPP.
- Guardar los respaldos en otra memoria.
- Hacer respaldos de otras memorias.
- Eliminar respaldos anteriores.

3. Funcionalidades adicionadas posteriormente

Conforme avanzaron los Sprint, se añadieron nuevas características para mejorar la versatilidad del programa:

- Comprimir las carpetas en formato .zip.
- **Modo “disco externo recuperado”**: al montar o conectar un disco externo (USB/HDD retirado de otra PC), el sistema buscará archivos no pertenecientes al sistema operativo.
- **Modo red interna**: posibilidad de extraer archivos de otra PC mediante la red local, implementando un modo servidor y un modo usuario.

Seguridad

Al ser un programa de copiado o recuperación de archivos, solo requiere **permisos de usuario**. No tiene acceso directo a Internet, ya que únicamente realiza copias dentro de una red interna, por lo que no demanda medidas de seguridad avanzadas. Lo importante es que se ejecute en una **red confiable y segura** para proteger la información del usuario.

Usabilidad

El sistema no requiere de gran capacitación, ya que el propio programa guía al usuario durante el proceso. Su **interfaz es sencilla** y proporciona **informes claros** sobre el resultado de la operación, indicando si la copia se realizó con éxito o si ocurrió algún error.

Disponibilidad

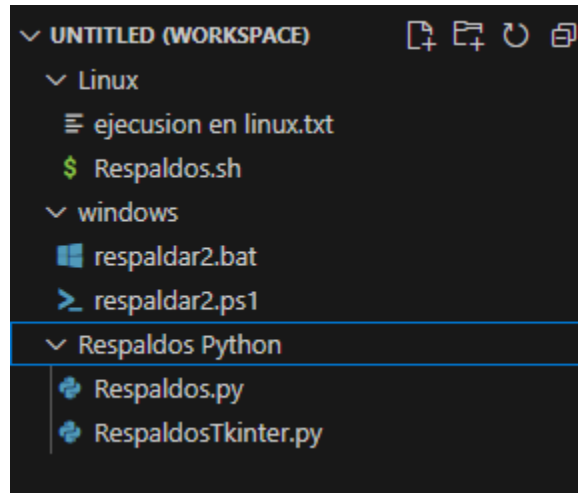
El usuario tendrá acceso al sistema con una **disponibilidad del 100%**, pudiendo utilizarlo en cualquier momento que lo necesite.

Creación del proyecto en Visual Studio Code

Para el desarrollo del sistema se utilizó **Visual Studio Code**, el cual permite trabajar con múltiples lenguajes de programación y facilita la instalación de extensiones. En este caso, únicamente fue necesario instalar los recursos de desarrollo de **Python**.

Se establecieron **tres espacios de trabajo**:

1. **Pruebas en Windows.**
2. **Pruebas en Linux.**
3. **Aplicación definitiva**, desarrollada en Python como versión multiplataforma.



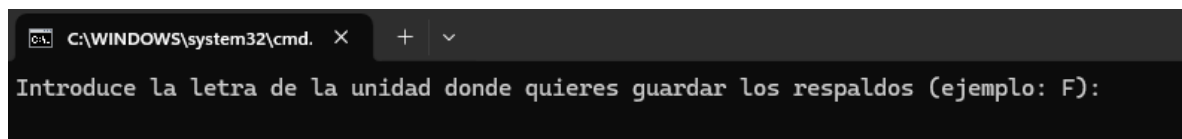
Sprint 1 – Planificación e Investigación

En este primer Sprint se definió qué elementos serían integrados en la aplicación, priorizando los puntos más importantes. Además, se inició la **investigación sobre el funcionamiento de los diferentes sistemas operativos**, con el objetivo de comprender cómo acceder y leer sus carpetas de manera eficiente.

Se realizó la investigación de cómo funcionan los distintos sistemas operativos en la lectura y organización de carpetas, con el fin de asegurar la compatibilidad del programa en varios entornos.

Sprint 2 – Diseño de la Aplicación

En este Sprint se trabajó en la **forma y diseño de la aplicación**, definiendo la manera en que el usuario interactuaría con el sistema. En esta etapa inicial, se pensó en un diseño sencillo, donde el usuario pudiera seleccionar opciones básicas. Durante esta etapa también se contemplaron las extensiones necesarias y las funcionalidades básicas que darían forma al prototipo inicial. Sin embargo, la complejidad surgió al desarrollar la versión en **Python**, ya que se buscó la forma más simple y ligera de implementarla sin perder funcionalidad.



```
C:\WINDOWS\system32\cmd. X + v
=====
Menu Respaldos
=====
1) Respaldo general (Documentos, Escritorio, Descargas, Imagenes)
2) Respaldo por tipo de extension (Archivos organizados en carpetas)
3) Respaldo de unidad extraible o disco (USB, disco duro, CD, disquete)
4) Respaldo solo carpeta C:\xampp\htdocs
5) Eliminar respaldos anteriores
6) Salir
Selecciona una opcion: |
```

Sprint 3 – Desarrollo e Implementación

En esta fase se programaron las funciones principales del sistema en Python, enfocándose en la recuperación de archivos por extensión, respaldos en memoria externa y eliminación de copias anteriores. Después de esto se añadieron mejoras como la compresión en formato .zip, el modo de disco externo recuperado y el uso de red interna mediante servidor y cliente.

Sprint 4 – Pruebas y Optimización

Finalmente, se realizaron pruebas en diferentes entornos (Windows y Linux) para validar la estabilidad del programa.

Conclusiones y Recomendaciones

Conclusiones

El desarrollo de esta herramienta representó un reto importante, ya que superó en ciertos momentos mis propias capacidades. La complejidad principal radicó en lograr que el programa fuera **multiplataforma**, debido a las diferencias significativas entre la estructura de carpetas de Windows y Linux.

Otro de los grandes desafíos fue mantener el sistema **ligero y eficiente**, evitando sobrecargarlo con funciones innecesarias. Hacer que un programa sea simple para el usuario, pero eficiente a nivel interno requiere escribir un código optimizado, lo cual aumenta el nivel de dificultad.

A pesar de estos retos, el proyecto permitió adquirir experiencia en el uso de distintos lenguajes y entornos de desarrollo, además de reforzar habilidades en **análisis de requisitos, diseño de interfaces, pruebas y optimización**. En conclusión, se logró crear una herramienta funcional que cumple con su propósito principal y que puede servir como base para futuras mejoras o implementaciones.

Recomendaciones

1. Optimización del código

Seguir mejorando la eficiencia del programa, evitando procesos innecesarios para que siga siendo ligero incluso en computadoras de bajos recursos.

2. Seguridad

Aunque el programa trabaja en redes internas, sería recomendable en futuras versiones integrar validaciones adicionales y mecanismos de seguridad para evitar accesos indebidos.

3. Interfaz de usuario

Incluir mejoras visuales y más opciones gráficas que faciliten la interacción, sin perder la simplicidad del sistema.

4. Compatibilidad

Ampliar la compatibilidad con más sistemas operativos o entornos, incluyendo versiones más recientes de Linux y MacOS.

5. Documentación

Mantener una guía de usuario clara y actualizada, para que cualquier persona pueda utilizar la aplicación sin necesidad de conocimientos técnicos avanzados.

6. Escalabilidad

Evaluar la posibilidad de añadir funciones avanzadas como respaldos en la nube o copias automáticas programadas.

Problemas Dentro de la Programación

Durante el desarrollo del sistema de respaldo se presentaron varios inconvenientes técnicos:

1. Bloqueo de carpetas protegidas:

En algunos equipos, ciertas carpetas del sistema se encuentran restringidas por permisos del sistema operativo, lo que provocaba que el programa se detuviera al intentar accederlas.

Para solucionar este problema, se implementó un **ciclo de respaldo con control de errores**, que permite continuar el proceso aun cuando una carpeta no pueda ser leída o copiada.

Sin embargo, esto ocasiona que **no se respalden todas las carpetas** del usuario, limitando parcialmente la cobertura.

2. Respaldo por extensión de archivo:

Esta modalidad permitió penetrar con mayor profundidad en las estructuras de carpetas, accediendo incluso a archivos almacenados en servicios como **OneDrive**.

No obstante, surgió el inconveniente de que el programa **copiaba archivos del sistema** (por ejemplo, elementos del inicio o de la barra de tareas), lo que resultaba innecesario e incluso riesgoso.

Para mitigar este problema se diseñó una **lista negra (blacklist)** que permite identificar y excluir carpetas no deseadas.

Además, se añadió una función que permite al usuario **eliminar manualmente** dichas carpetas detectadas después del respaldo, ofreciendo un mayor control sobre el contenido final.

Modificaciones

Con base en las pruebas realizadas, se introdujeron mejoras para optimizar el rendimiento y la seguridad del respaldo:

- **Lista negra (Blacklist):**

Contiene rutas o nombres de carpetas que deben ser **excluidas automáticamente**, como directorios del sistema, carpetas de instalación de programas o archivos temporales.

- **Lista blanca (Whitelist):**

Define carpetas **seguras o prioritarias** para garantizar que los datos personales del usuario —como documentos, imágenes, proyectos o archivos de programación— siempre sean respaldados, incluso si se encuentran en ubicaciones no estándar.

- **Mecanismo de detección inteligente:**

El sistema verifica continuamente las rutas de respaldo y ajusta las listas blanca y negra en función de las carpetas encontradas, mejorando su desempeño con cada ejecución.

Resultados

Gracias a estas modificaciones, el programa ahora:

- Evita errores por permisos o carpetas bloqueadas.
- Excluye de forma automática archivos innecesarios del sistema.
- Prioriza los datos personales del usuario.
- Mejora progresivamente su precisión en los respaldos a través del uso de listas dinámicas.

Características específicas para macOS

Se pudo conseguir un equipo Mack y resulto no funcionar así que tuvimos que buscar la forma de hacerlo lamentablemente no pudimos hacer las pruebas pertinentes, pero esperamos que haya servido todo lo buscado para que sea funcional

El sistema de respaldos ha sido adaptado para garantizar compatibilidad completa con **macOS**, considerando sus rutas, permisos y estructura de archivos particular.

Rutas bloqueadas específicas de macOS

Se excluyen automáticamente las siguientes carpetas y archivos del sistema, ya que contienen configuraciones internas o datos de caché:

Ruta	Descripción
.Trash	Papelera del usuario
.Spotlight-V100	Índice de búsqueda de Spotlight
.fsevents	Registro de eventos del sistema de archivos
Caches	Cachés del sistema
Logs	Archivos de registro del sistema

Estas rutas son ignoradas por el sistema de respaldo para **evitar errores de permisos y reducir el tamaño innecesario** del respaldo.

carpetas de usuario en macOS

El sistema detecta automáticamente las carpetas de usuario más comunes:

- ~/Documents – Documentos
- ~/Downloads – Descargas
- ~/Desktop – Escritorio

- ~/Pictures – Imágenes
- ~/Music – Música
- ~/Movies – Videos
- ~/Public – Archivos compartidos
- ~/Library – Configuraciones y datos de aplicaciones
- ~/Development – Proyectos de desarrollo

Estas rutas son consideradas **prioritarias** dentro del respaldo general y en los respaldos por extensión.

Rutas de desarrollo en macOS

Para desarrolladores, el sistema incluye detección de rutas de herramientas de programación y entornos de desarrollo:

Ruta	Descripción
~/Library/Developer	Archivos generados por Xcode
~/Library/MobileDevice	Sincronización con dispositivos iOS
~/Library/Application Support	Configuraciones de aplicaciones
/Applications/XAMPP/xamppfiles/htdocs	Carpeta raíz de proyectos XAMPP
/Applications/XAMPP/htdocs	Alternativa de ruta XAMPP
~/Applications/XAMPP/xamppfiles/htdocs	XAMPP instalado localmente por usuario

Extensiones específicas de macOS

El sistema reconoce y respalda archivos con extensiones usadas frecuentemente en el ecosistema Apple:

- .m, .mm — Código fuente en Objective-C
- .plist — Archivos de preferencias de sistema o apps
- .strings — Archivos de localización e internacionalización

Recomendaciones Externas

Las recomendaciones de mejoras pueden ser mandadas a el correo o directamente con su servidor

Una de ellas con mención fue Comprimir las carpetas en formato .zip

Evaluaciones de calidad

Las evaluaciones de calidad, aunque no son formales nos sirven para mejorar nuestro programa este fue hecho por algunos compañeros que no saben sobre este programa se les pidió que fueran severos ya que lo que nos interesa es que este proyecto se quede como practica y base para ir mejorando

Rodríguez Martínez Gerardo Uriel

Según lo informado por esta persona, requiere atención en la funcionalidad de eliminar, ya que, si alguien mete un archivo a esta carpeta, aparece en la lista y puede ser eliminado. La sugerencia es —y cito— **“se sugiere que podría solucionarse al filtrar el contenido de la carpeta por medio del patrón de texto (expresión regular) con el que se asignan los nombres a los respaldos, ya qué es un patrón constante con todos los respaldos generados.”**

También hizo la siguiente observación —y cito— **“cuando se cambia la ruta de los respaldos, por ejemplo, la primera vez se deja la opción default, que es la de guardarlos en la carpeta Respaldos, y si en una futura sesión se ingresa una ruta diferente no va mostrar los respaldos que se guardaron en Respaldos, no es un error crítico ni mucho menos, pero sí un área de mejora en guardar la configuración persistente del programa la ruta donde se realizan los respaldos.”**

En esta parte, estaba pensando en dejar un archivo **.txt** que guarde la ruta donde se guardó y que indique que se hizo un respaldo y que este se encuentra en dicha ruta.

También mencionó que, si se interrumpe el proceso y no hay forma de retomarlo, podemos integrar una manera en la que se mande un error y se active un botón o un apartado para reanudar.

Además, indicó que en lugar de mostrar las rutas que se van respaldando, se coloque una barra de carga. Sin embargo, estaba pensando en poder activar o desactivar la creación de un **.txt** con las rutas copiadas.

Debemos revisar si es posible hacer más eficiente el proceso de respaldo, ya que indica que solo respalda elementos pequeños y le gustaría que fuera más eficiente, aunque no lo sugirió explícitamente.

Finalmente, se debe mejorar el mensaje de error para que sea comprensible para un usuario sin conocimientos técnicos.

Conclusión

Mi conclusión de esta evaluación de calidad es que se requieren diversas mejoras para optimizar la experiencia del usuario y el funcionamiento del sistema de respaldos. En primer lugar, es necesario restringir la eliminación exclusivamente a los respaldos generados por el programa. Además, se propone almacenar en un archivo **.txt** la ruta donde se guardan los respaldos, para que el usuario pueda consultar dicha información incluso si no se encuentran en la carpeta actual. Esta medida sustituiría la visualización en terminal, dando paso a una barra de progreso más intuitiva.

Asimismo, se considera útil incorporar un botón que permita reanudar un respaldo en caso de una interrupción inesperada, junto con la mejora de los mensajes de error para que usuarios sin conocimientos técnicos comprendan fácilmente la situación. Finalmente, se recomienda trabajar en la optimización del proceso de respaldo para hacerlo más eficiente.

Referencias Bibliográficas

<https://www.tenstep.ec/portal/articulos-boletin-tenstep/41-Scrum/253-Scrum-como-escribir-historias-de-usuarios-sin-morir-en-el-intento>.

[Visual Studio Code - Code Editing. Redefined](#)

[¿Qué es Python y para qué se usa? Guía para principiantes | Coursera](#)

[Explicación del shell Bourne de Linux](#)

[¿Qué es PowerShell y para qué sirve? - Definición](#)

[Qué es un archivo bat o batch, ejemplos y usos prácticos](#)